

SUPPLY CHAIN DEMAND FORECASTING

Submitted by

ANU SHREE KA

INTRODUCTION

Demand forecasting predicts future product requirements from historical and contextual information. It is especially important in retail, FMCG, logistics and manufacturing, where inaccurate forecasts can cause stockouts or excess inventory. The internship guide proposes a multi-horizon forecasting system using sales history, promotions, holidays, inventory, competitor pricing, web trends and macroeconomic data. It identifies TFT, LSTM and SHAP as advanced technologies.

- . The student baseline uses historical sales lags and rolling demand.
- . The system can later incorporate promotions, holidays and external variables.
- . Forecasts support planning; they do not replace business judgment.

. **INDUSTRY SCENARIO**

The guide describes an FMCG retailer with hundreds of SKUs and multiple distribution centres. Popular products can suffer

stockouts while slow-moving products accumulate excess inventory. Demand also changes because of promotions, holidays, competitor prices and external signals. An AI forecasting system should therefore learn temporal patterns and make forecasts at multiple horizons.

- Retail and FMCG: SKU-level forecasting.
- Pharmaceuticals: cold-chain inventory and regional demand.
- Manufacturing: raw-material requirements planning.
- Logistics: warehouse staffing and route optimization.
- E-commerce: fulfilment-centre inventory allocation

PROBLEM STATEMENT

The project aims to estimate future demand accurately enough to improve inventory decisions. The guide requires sales history and contextual variables, multi-horizon forecasting, support for static and dynamic features, interpretability and probabilistic outputs. Its performance target is a 15–20% reduction in WMAPE compared with a baseline. The simplified implementation demonstrates the fundamental workflow using a Random Forest baseline.

- Inputs can include sales, promotion, holidays, inventory and competitor prices.

- Outputs can be point forecasts and, in an advanced system, prediction intervals.
- The model should be evaluated on future observations rather than randomly mixed data.

OBJECTIVES

.

The objectives are to clean historical sales, engineer time-dependent features, train a forecasting model, evaluate forecast errors and provide an interactive prediction interface.

The project also establishes a foundation for later upgrades to LSTM, XGBoost and Temporal Fusion Transformer models.

- Create lag and rolling features.
- Use a chronological train/test split.
- Calculate MAE and RMSE; add WMAPE for the advanced version.
- Save the trained model for inference.
- Provide a Streamlit dashboard.

PROPOSED SYSTEM

The proposed pipeline is: sales history → date processing → feature engineering → chronological split → forecasting model → evaluation → model artifact → interactive dashboard. A production version would also ingest promotion calendars, holiday dates, inventory levels, competitor pricing, web trends and macroeconomic indicators. The guide proposes AWS services for deployment and monitoring.

- Historical data provides the learning signal.

- Feature engineering converts time-series history into model inputs.
- The model produces a future-demand estimate.
- Monitoring checks forecast accuracy and data drift.

• **SYSTEM ARCHITECTURE**

The data layer contains historical observations. The feature layer creates lag, rolling and calendar variables. The model layer learns the relationship between those features and sales. The evaluation layer compares forecasts with actual future

values. Finally, the application layer exposes forecasts through a dashboard or FastAPI endpoint. Advanced deployments can store data and model artifacts in S3 and monitor the service through CloudWatch.

- Data → features → model → metrics → deployment.
- Optional SHAP explains important forecast drivers.
- Automated retraining can be triggered as new data arrives.

DATASET & DATA REQUIREMENTS

The guide lists the M5 Forecasting competition dataset with around 42,000 SKUs, the Favorita grocery sales dataset and UCI time-series datasets. The simplified program expects sales.csv with date, product and sales columns. A richer implementation should add promotional calendar, holiday dates, web search trends, inventory levels, competitor pricing and macroeconomic data.

- Date identifies the time point.
- Product identifies the SKU or item series.
- Sales is the target variable.

- Additional external variables improve the ability to model business events.

PREPROCESSING

Dates are converted to a standard datetime representation and observations are sorted by product and date. Lag features are calculated using only previous observations. A rolling mean is shifted so that the current target does not leak into its own feature. Rows without enough historical context are removed. This process is essential because time-series leakage can make evaluation appear much better than real-world performance.

- Convert date values.
- Sort by product and time.
- Create lag_1 and lag_7.
- Create a previous seven-observation rolling mean.
- Drop rows without sufficient history.

FEATURE ENGINEERING

Lag features describe recent demand. lag_1 captures the previous observation, while lag_7 provides a weekly-style historical reference when the data is daily. Rolling averages reduce short-term noise. In the advanced guide, features can include promotions, holidays, competitor prices, inventory, web trends and macroeconomic indicators. Calendar variables such as day of week and month can also capture recurring patterns.

- Lag features: recent history.
- Rolling statistics: local trend/smoothing.
- Calendar features: seasonality.

- External features: business events and market conditions.

FORECASTING ALGORITHM

The student implementation uses Random Forest regression as a baseline. An ensemble of decision trees can model non-linear relationships between recent demand features and the target. The guide's advanced roadmap proposes ARIMA/SARIMA baselines, LSTM, XGBoost and ultimately Temporal Fusion Transformer. TFT is especially suited to mixed static and time-varying inputs and provides built-in interpretability mechanisms.

- Random Forest: simple non-linear baseline.

- LSTM: sequence-oriented neural network.
- XGBoost: strong tabular forecasting baseline.
- TFT: advanced multi-horizon architecture

TRAINING & EVALUATION

The data is split chronologically, with earlier observations used for training and later observations used for testing. The model is trained only on the training portion. Predictions on the test portion are compared with actual sales. MAE gives the average absolute error and RMSE gives greater weight to large errors.

The guide also recommends WMAPE for business-oriented comparison.

- Chronological split prevents future leakage.
- MAE is easy to interpret in sales units.
- RMSE highlights large forecast mistakes.
- WMAPE is useful for comparing errors across demand volumes.

OUTPUT & RESULTS

A real run prints the number of training and testing rows, MAE, RMSE and a model-save confirmation. The Streamlit dashboard displays a predicted number of units for the entered recent-demand values. Because the actual sales.csv dataset is not part of the internship guide, numerical metrics cannot be honestly filled in without selecting and running a dataset. The guide's 15–20% WMAPE improvement is a target for an industry-grade version, not a fabricated result.

- Console: training rows, testing rows, MAE, RMSE.
- File: demand_model.joblib.

- Dashboard: predicted demand in units.
- Advanced report: compare against a baseline and show WMAPE.

COMPLETE CORE/TRAINING CODE

The following is the complete implementation used for the core training or RAG-retrieval stage. It is intentionally kept as code, while the other slides explain the logic in presentation-friendly detail.

```
# train.py import
pandas as pd import
numpy as np import
joblib from
sklearn.ensemble
import
RandomForestRegresso
r from
sklearn.metrics
```

```
import
mean_absolute_error,
mean_squared_error

df=pd.read_csv(
"sales.csv")
df["date"]=pd.to_datetime(df["date"])
df=df.sort_values(["product","date"])

group=df.groupby("product")["sales"]
```

```
df["lag_1"]=group.shift(1)
df["lag_7"]=group.shift(7)
df["rolling_7"]
=group.shift(1)
.rolling(7).mean()
df=df.dropna()

features=["lag_1","lag_7","rolling_7"]
cut=int(len
```

```
(df)*0.80)
train=df.iloc[
oc[:cut]
test=df.iloc[
c[cut:]

model=Random
ForestRegres
sor(
n_estimators
=200,
random_state
=42,
n_jobs=-1 )
model.fit(tr
```

```
ain[features
],train["sal
es"])

pred=model.pred
ict(test[featur
es])

mae=mean_absolu
te_error(test["
sales"],pred)

rmse=np.sqrt(me
an_squared_erro
r(test["sales"]
,pred))
```



```
print("Trainin  
g  
rows:",len(tra  
in))  
print("Testing  
rows:",len(tes  
t))  
print("MAE:",r  
ound(mae,2))  
print("RMSE:",  
round(rmse,2))  
joblib.dump(mod  
el,"demand_mode  
l.joblib")
```

```
print("Model  
saved as  
demand_model.jo  
bilib")
```

COMPLETE APPLICATION CODE

This is the complete application/inference interface. It loads the saved model or index and exposes the project through a simple user interface.

```
#
app.py
import
stream
lit as
st
import
pandas
as pd
```

```
import
joblib

st.set_page_con
fig(page_title=
"Demand
Forecasting")
st.title("Suppl
y Chain Demand
Forecasting")

model=joblib.lo
ad("demand_mode
l.joblib")
```

```
lag1=st.number_input("Previous-day  
demand",0.0,100000.0,100.0  
)  
lag7=st.number_input("Demand 7 days  
ago",0.0,100000.0,95.0)  
rolling=st.number_input("7  
-day rolling  
average",0.0,100000.0,98.0  
)  
  
if st.button("Forecast  
Demand"):  
x=pd.DataFrame(
```

```
[[lag1,lag7,rolling]],  
columns=["lag_1","lag_7",  
"rolling_7"]  
forecast=model.predict(x)  
[0]  
st.success(f"Predicted  
demand: {forecast:.0f}  
units")  
st.write("Use  
the forecast as a  
planning aid and compare  
it with actual demand.")
```

COMPLETE OUTPUT & RESULT INTERPRETATION

A real execution prints training rows, testing rows, MAE and RMSE, followed by confirmation that `demand_model.joblib` was saved. The dashboard accepts `lag_1`, `lag_7` and `rolling_7` and returns a predicted number of units. The guide's 15–20% WMAPE reduction is a performance target for the advanced system, not a measured result of this baseline.

- Console output: Training rows and Testing rows.
- Console output: MAE and RMSE.

- Saved artifact: demand_model.joblib.
- Dashboard output: predicted demand in units.
- Final submission should include a screenshot from the actual run.

APPLICATIONS

The guide lists retail and FMCG SKU forecasting, pharmaceutical cold-chain demand, manufacturing raw-material planning, logistics staffing and route optimization, and e-commerce fulfilment allocation. Forecasts can support purchasing and inventory planning when combined with operational constraints.

LIMITATIONS & FUTURE SCOPE

The baseline uses only lag and rolling features, so it does not represent the full guide requirements. Future versions should add promotions, holidays, competitor prices, inventory, web trends and macroeconomic variables. The guide proposes LSTM, XGBoost and TFT, plus SHAP explainability, automated retraining, drift detection and MLOps deployment.

CONCLUSION

The project demonstrates how historical demand can be converted into predictive features and used to train a forecasting model. It provides a manageable baseline before moving to multi-horizon TFT forecasting. The final report should compare the model with a simple baseline and report MAE, RMSE and WMAPE from a real dataset.

References

Primary source: AI Internship Projects Guide, 2025 edition. The guide lists M5 Forecasting, Favorita and UCI datasets, TFT/LSTM/XGBoost technologies, SHAP and AWS deployment

options. Additional reference: [scikit-learn RandomForestRegressor documentation](#).